

CHAPTER - 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The Heads Up For Tails (HUFT) E-Commerce Order Management Database System is designed to efficiently manage customers, products, orders, payments, shipments, and administrators. Functional dependencies help identify the relationship between attributes in each table, while redundancy analysis and data anomaly detection ensure data consistency and reduce duplication. A well-designed database improves performance, maintains data integrity, and supports efficient order management.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID $\rightarrow$ Customer_Name, Email, Phone_Number, Address

Redundancy Analysis

Customer information is stored only once in the Customer table. Orders store only the Customer_ID, preventing duplicate customer records.

Data Anomaly:

No data anomalies are present because customer information is stored only once in the Customer table. This prevents insert, update, and delete anomalies and ensures data consistency.

2. CATEGORY TABLE

Functional Dependency

Category_ID $\rightarrow$ Category_Name

Redundancy Analysis

Each category is stored only once. Products reference the category using Category_ID instead of repeating the category name.

Data Anomaly:

No data anomalies are present because category details are maintained in a separate table. This avoids duplication and ensures that updates to category information are made only once.

3. PRODUCT TABLE**Functional Dependency**

Product_ID $\rightarrow$ Product_Name, Price, Stock, Category_ID

Redundancy Analysis

Product details are stored only in the Product table. Order details use Product_ID instead of repeating product information.

Data Anomaly:

No data anomalies are present because product information is stored independently and linked to categories using foreign keys. This prevents duplicate product records and maintains data integrity.

4. ORDERS TABLE**Functional Dependency**

Order_ID $\rightarrow$ Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Orders table stores only order-related information. Customer and product details are referenced using foreign keys.

Data Anomaly:

No data anomalies are present because order details are stored separately and linked to customers through foreign keys. This design prevents data inconsistency and unnecessary duplication.

5. ORDER_ITEM TABLE

Functional Dependency

Order_Item_ID $\rightarrow$ Order_ID, Product_ID, Quantity, Price

Redundancy Analysis

This table connects orders and products, avoiding repeated product information in the Orders table.

Data Anomaly:

No data anomalies are present because the Order_Item table stores only the relationship between orders and products. This avoids duplicate product information and supports accurate order management.

6. PAYMENT TABLE

Functional Dependency

Payment_ID $\rightarrow$ Order_ID, Payment_Method, Payment_Status

Redundancy Analysis

Payment information is stored separately from the Orders table to avoid duplicate payment records.

Data Anomaly:

No data anomalies are present because payment details are maintained in a separate table. This ensures that payment information can be updated or deleted without affecting order records.

7. SHIPMENT TABLE

Functional Dependency

Shipment_ID $\rightarrow$ Order_ID, Tracking_Number, Delivery_Status

Redundancy Analysis

Shipment details are maintained in a separate table, preventing repeated delivery information.

Data Anomaly:

No data anomalies are present because shipment details are stored independently from order information. This prevents data duplication and allows shipment records to be managed efficiently.

8. ADMIN TABLE**Functional Dependency**

Admin_ID $\rightarrow$ Admin_Name, Email, Password

Redundancy Analysis

Administrator information is stored separately to avoid repeating login details across different tables.

Data Anomaly:

No data anomalies are present because administrator information is stored only once in the Admin table. This eliminates duplicate records and ensures easy maintenance of administrator details.

CONCLUSION

The HUFT E-Commerce Order Management Database System is designed using proper functional dependencies and normalized tables. Each entity stores only its relevant data, reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains integrity, and ensures efficient database management for the e-commerce system.